DS Modules' Solution

Module 1

What is Data Structures?

A data structure is a specialized format for organizing, processing, retrieving and storing data. While there are several basic and advanced structure types, any data structure is designed to arrange data to suit a specific purpose so that it can be accessed and worked with in appropriate ways. Most importantly, data structures frame the organization of information so that machines and humans can better understand it. In computer science and computer programming, a data structure may be selected or designed to store data for the purpose of using it with various algorithms. In some cases, the algorithm's basic operations are tightly coupled to the data structure's design. Each data structure contains information about the data values, relationships between the data and -- in some cases -- functions that can be applied to the data.

Abstract Data Type(ADT)

- Abstract Data type (ADT) is a type (or class) for objects whose behavior is defined by a set of values and a set of operations.
- It does not specify how data will be organized in memory and what algorithms will be used for implementing the operations.
- It is called "abstract" because it gives an implementation-independent view.
- To process data with a computer, we need to define the data type and the operation to be performed on the data.

- The definition of the data type and the definition of the operation to be applied to the data is part of the idea behind an abstract data type (ADT)
- ADT means to hide how the operation is performed on the data.
- In other words, the user of an ADT needs only to know that a set of operations are available for the data type, but does not need to know how they are applied.

Simple Abstract Data Type

- Many programming languages already define some simple ADTs as integral parts of the language.
- For example, the C language defines a simple ADT as an integer. The type of this ADT is an integer with predefined ranges.
- C also defines several operations that can be applied to this data type (addition, subtraction, multiplication, division and so on).
- C explicitly defines these operations on integers and what we expect as the results.
- A programmer who writes a C program to add two integers should know about the integer ADT and the operations that can be applied to it.

Types of Data Structures

1. Linear Data Structure

- It is a type of data structure where the arrangement of the data follows a linear trend.
- The data elements are arranged linearly such that the element is directly linked to its previous and the next elements.
- Implementation of the linear structure of data in a computer's memory is easy as the data is organized sequentially.
- The data elements stored in the data structure have only one relationship.
- Traversal of the data elements can be carried out in a single run as the data elements are stored in a single level.
- Array, queue. Stack, linked list, etc. are examples of this type of structure.

2. Non Linear Data Structure

- Data structures where data elements are not arranged sequentially or linearly are called non-linear data structures.
- In a non-linear data structure, single level is not involved.
- Therefore, we can't traverse all the elements in single run only.
- Non-linear data structures are not easy to implement in comparison to linear data structure
- It utilizes computer memory efficiently in comparison to a linear data structure.
- Its examples are trees and graphs.

Module 2

What is Stack?

- Stack is a UFO (Last In First Out) structure.
- It is an ordered list of the same type of elements.
- A stack is a linear list where all insertions and deletions are permitted only at one end of the list.
- When elements are added to stack it grows at one end.
- Similarly, when elements are deleted from a stack, it shrinks at the same end.
- Consider an example of plates stacked over one another in the canteen.
- Applications of Stack
  1. Expression conversion
  (a) Infix to postfix
  (b) Infix to prefix (d) Prefix to infix
  (c) Postfix to infix
  2. Expression evaluation
  3.Parsing
  4.Simulation of recursion
  5. Function call

What is Queue?

- It is a special kind of list, where items are inserted at one end (the rear) and deleted from the other end (front).
- Queue is a FIFO (First In First Out) list.
- It is used for temporary storage of data
- We come across the term queue in our day to day life.

- We see a queue at a railway reservation counter, or a movie theatre ticket counter. Before getting the service, one has to wait in the queue. After receiving the service, one leaves the queue. Service is provided at one end (the front) and people join at the other end (rear).
- Operations on Queue
  1. Insertion:
  Placing an item in a queue is called "insertion or enqueue", which is done at the end of the queue called "rear".
  2.Deletion:
  Removing an item from a queue is called "deletion or dequeue", which is done at the other end of the queue called "front".

Types of Queues:

1. Dequeue
   - Deque stands for double ended queue.
   - Elements can be inserted or deleted at either end.
   - Also known as head-tail linked list.
   -
2. Circular Queue
   - Circular queue are used to remove the drawback of simple queue.
   - Both the front and the rear pointers wrap around to the beginning of the array.
   - It is also called as "Ring buffer".
3. Priority Queue
   - It is collection of elements where elements are stored according to the their priority levels.

- Inserting and removing of elements from queue is decided by the priority of the elements.
- An element of the higher priority is processed first.
- Two element of same priority are processed on first-come-first-served basis

Applications of Queue

- Semaphores
- FCFS (first come first serve) scheduling, example: FIFO queue
- Spooling in printers
- Buffer for devices like keyboard
- CPU Scheduling
- Memory management
- Applied on Operating system to handle the interruption.
- Traffic software

What is Dequeue?

The deque stands for Double Ended Queue. Deque is a linear data structure where the insertion and deletion operations are performed from both ends. We can say that deque is a generalized version of the queue.

There are two types of deque –

- Input restricted Queue

In input restricted queue, insertion operation can be performed at only one end, while deletion can be performed from both ends.

- Output restricted Queue

In output restricted queue, deletion operation can be performed at only one end, while insertion can be performed from both ends.

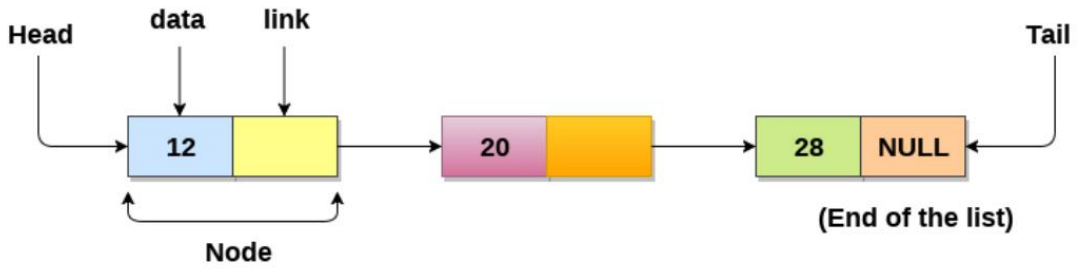
Operations performed on deque:

1. Insertion at front
2. Insertion at rear
3. Deletion at front
4. Deletion at rear

Module 3

Linked List

- Linked List can be defined as collection of objects called nodes that are randomly stored in the memory.
- A node contains two fields i.e. data stored at that particular address and the pointer which contains the address of the next node in the memory.
- The last node of the list contains pointer to the null.
- The list is not required to be contiguously present in the memory.
- It is also a sequence of links which contains items. Each link contains a connection to another link.
- The node can reside any where in the memory and linked together to make a list. This achieves optimized utilization of space.
- list size is limited to the memory size and doesn't need to be declared in advance.
- Empty node can not be present in the linked list.
- We can store values of primitive types or objects in the singly linked list.
- Linked list is the second most-used data structure after array



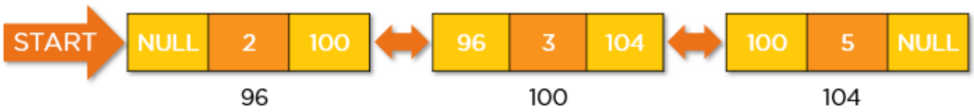
Types of Linked list

- Singly linked lists

  A singly linked list is a unidirectional linked list. So, you can only traverse it in one direction, i.e., from head node to tail node.

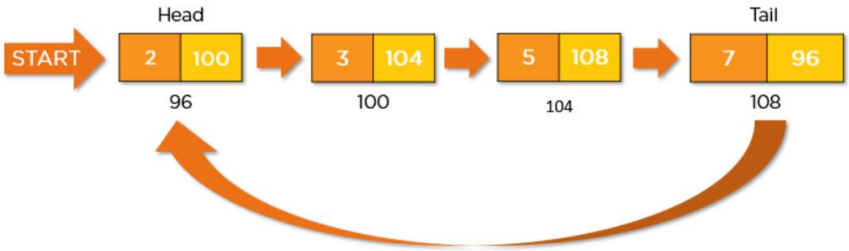
- Doubly Linked list

  A doubly linked list is a bi-directional linked list. So, you can traverse it in both directions. Unlike singly linked lists, its nodes contain one extra pointer called the previous pointer. This pointer points to the previous node.

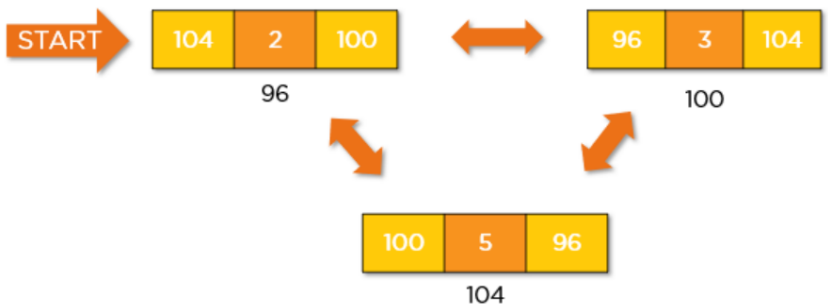


- Circular Linked List

  A circular Linked list is a unidirectional linked list. So, you can traverse it in only one direction. But this type of linked list has its last node pointing to the head node. So while traversing, you need to be careful and stop traversing when you revisit the head node.



- Doubly Circular Linked List
  A circular doubly linked list is a mixture of a doubly linked list and a circular linked list. Like the doubly linked list, it has an extra pointer called the previous pointer, and similar to the circular linked list, its last node points at the head node. This type of linked list is the bi-directional list. So, you can traverse it in both directions.



(Singly Linked List Application) What is polynomial? and its representation

A polynomial p(x) is the expression in variable x which is in the form

(ax n + bx n-1 + …. + jx+ k), where a, b, c …., k fall in the category of real numbers and 'n' is non negative integer, which is called the degree of polynomial.

An essential characteristic of the polynomial is that each term in the

polynomial expression consists of two parts:

- one is the coefficient
- other is the exponent

Example

$10x^2$ + 26x, here 10 and 26 are coefficients and 2, 1 is its

exponential value.

Array	Linked list
An array is a grouping of data elements of equivalent data type.	A linked list is a group of entities called a node. The node includes two segments: data and address.
It stores the data elements in a contiguous memory zone.	It stores elements randomly, or we can say anywhere in the memory zone.
In the case of an array, memory size is fixed, and it is not possible to change it during the run time.	In the linked list, the placement of elements is allocated during the run time.
The elements are not dependent on each other.	The data elements are dependent on each other.
The memory is assigned at compile time	The memory is assigned at run time.
It is easier and faster to access the element in an array.	In a linked list, the process of accessing elements takes more time.
In the case of an array, memory utilization is ineffective.	In the case of the linked list, memory utilization is effective.
When it comes to executing any operation like insertion, deletion, array takes more time.	When it comes to executing any operation like insertion, deletion, the linked list takes less time.
Works with static memory.	Works with dynamic memory.
They use contiguous memory locations.	They use non-contiguous memory locations.

Explain binary search with example.

Binary search is an efficient searching algorithm with only the requirement that the elements in the array are sorted already. It is a great implementation of the divide and conquer algorithm technique. It divides the array at the middle and then selects the first or the second half based on the search element and then repeats the process with the selected half until the search element is found (or not).

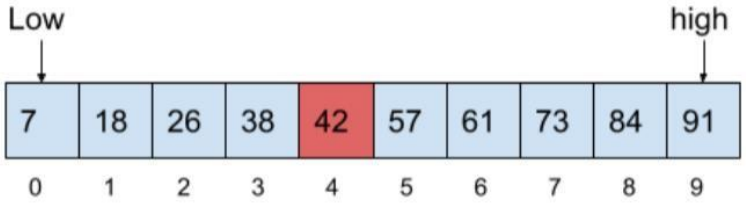
1. Works on the Divide and conquer technique to search for the elements.

2. Efficient than linear search algorithm.

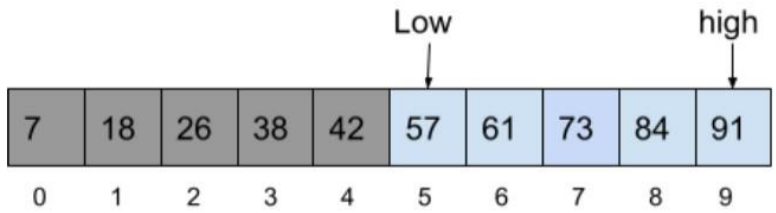
Objective: Search of element '73' in the below array

7	18	26	38	42	57	61	73	84	91
0	1	2	3	4	5	6	7	8	9

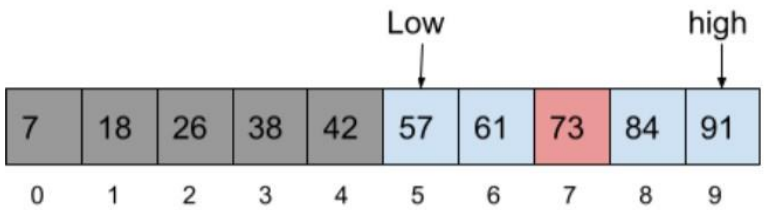
Step 1: Find the middle element.
mid = low + (high – low) / 2



Step 2: Select the second half by changing low to mid+1.
low=mid+1



Step 3: Repeat steps 1 and 2 until the element is found.



The element found at location 8.

Explain linear search with example.

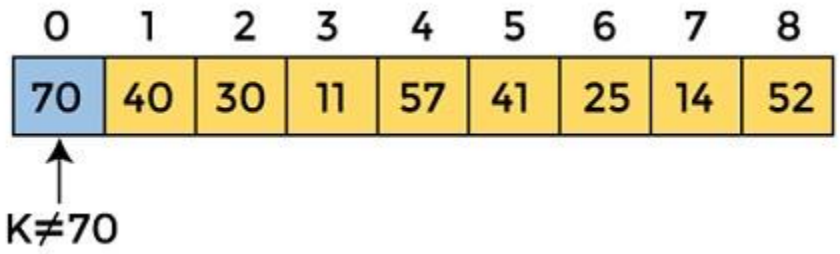
Linear search is a very simple search algorithm. In this type of search, a sequential search is made over all items one by one. Every item is checked and if a match is found then that particular item is returned, otherwise the search continues till the end of the data collection. Linear search is also called as sequential search algorithm. It is the simplest searching algorithm. In Linear search, we simply traverse the list completely and match each element of the list with the item whose location is to be found. If the match is found, then the location of the item is returned; otherwise, the algorithm returns NULL.

Let the elements of array are -

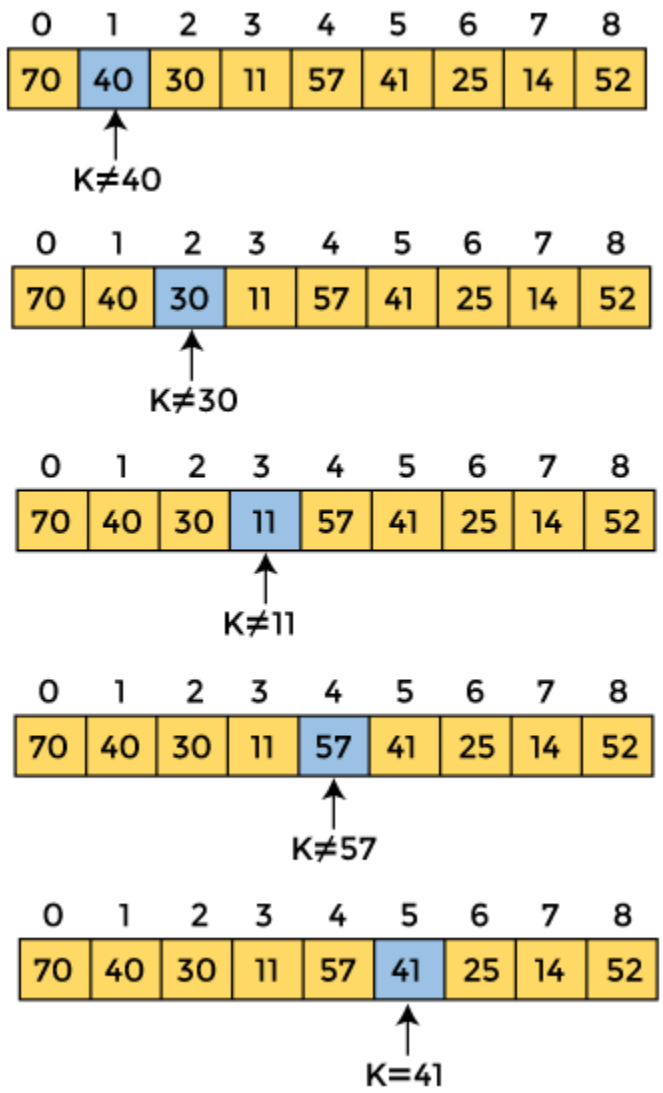
0	1	2	3	4	5	6	7	8
70	40	30	11	57	41	25	14	52

Let the element to be searched is **K = 41**

Now, start from the first element and compare **K** with each element of the array.



The value of **K,** i.e., **41,** is not matched with the first element of the array. So, move to the next element. And follow the same process until the respective element is found.

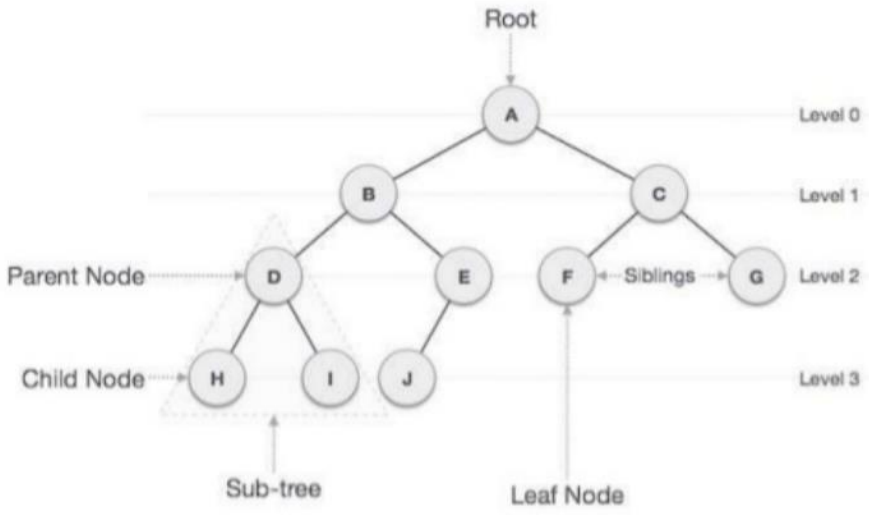


Now, the element to be searched is found. So algorithm will return the index of the element matched.

Module 3

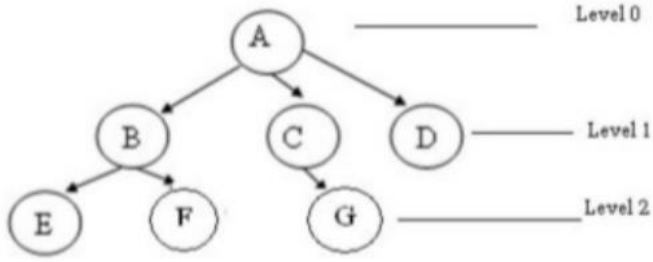
What is a Tree?

- A tree data structure is defined as a collection of objects or entities known as nodes that are linked together to represent or simulate hierarchy.
- A tree data structure is a non-linear data structure because it does not store in a sequential manner
- Trees are mainly used to represent data containing a hierarchical relationship between elements, for example, records, family trees and table of contents.
- Consider a parent-child relationship
- A tree is an abstract model of a hierarchical structure that consists of nodes with parent child relationship.
- Tree has a starting node called root node
- Every node other than root node is called parent node
- Nodes may have any number of parents



Some Key Terms:

- Root - Node at the top of the tree is called root.
- Parent - Any node except root node has one edge upward to a node called parent.
- Child - Node below a given node connected by its edge downward is called its child node.
- Sibling - Child of same node are called siblings
- Leaf - Node which does not have any child node is called leaf node.
- Sub tree - Sub tree represents descendants of a node.
- Levels - Level of a node represents the generation of a node. If root node is at level 0, then its next child node is at level 1, its grandchild is at level 2 and so on.
- keys - Key represents a value of a node based on which a search operation is to be carried out for a node.
- Degree of a node: The degree of a node is the number of children of that node
- Degree of a Tree: The degree of a tree is the maximum degree of nodes in a given tree
- Path: It is the sequence of consecutive edges from source node to destination node.
- Height of a node: The height of a node is the max path length form that node to a leaf node.
- Height of a tree: The height of a tree is the height of the root
- Depth of a tree: Depth of a tree is the max level of any leaf in the tree



- ✔ A is the root node
- ✔ B is the parent of E and F
- ✔ D is the sibling of B and C
- ✔ E and F are children of B
- ✔ E, F, G, D are external nodes or leaves
- ✔ A, B, C are internal nodes
- ✔ Depth of F is 2
- ✔ the height of tree is 2
- ✔ the degree of node A is 3
- ✔ The degree of tree is 3

Characteristics of trees

- Non-linear data structure
- Combines advantages of an ordered array Searching as fast as in ordered array
- Insertion and deletion as fast as in linked list
- Simple and fast

Application of Trees

- Directory structure of a file store Structure of an arithmetic expressions
- Used in almost every 3D video game to determine what objects need to be rendered.
- Used in almost every high-bandwidth router for storing router-tables.
- used in compression algorithms, such as those used by the .jpeg and .mp3 file- formats.

Binary Tree

Binary Tree is defined as a tree data structure where each node has at most 2 children. Since each element in a binary tree can have only 2 children, we typically name them the left and right child. A Binary tree is represented by a pointer to the topmost node (commonly known as the “root”) of the tree. If the tree is empty, then the value of the root is NULL.

Application of Binary tree:

1. In compilers, Expression Trees are used which are applications of binary trees.
2. Represent hierarchical data.
3. used in editing software like Microsoft Excel and spreadsheets.
4. useful for indexing segmented at the database is useful in storing cache in the system,
5. used to enable fast memory allocation in computers.
6. to perform encoding and decoding operations.

Binary Search Tree

- A Binary Search Tree (BST) is a special type of binary tree in which the left child of a node has a value less than the node’s value and the right child has a value greater than the node’s value.
- This property is called the BST property and it makes it possible to efficiently search, insert, and delete elements in the tree.

Binary tree data structure that has the following properties:

- The left subtree of a node contains only nodes with keys lesser than the node’s key.

- The right subtree of a node contains only nodes with keys greater than the node's key.
- This means everything to the left of the root is less than the value of the root and everything to the right of the root is greater than the value of the root. Due to this performing, a binary search is very easy.
- The left and right subtree each must also be a binary search tree.
- There must be no duplicate nodes(BST may have duplicate values with different handling approaches)

Operations on Binary Search Tree

The four basic operations of BST:

1. Searching
2. Insertion
3. Deletion
4. Traversals

- Searching
  Searching in BST involves the comparison of the key values. If the key value is equal to root key then, search successful, if lesser than root key then search the key in the left subtree and if the key is greater than root key then search the key in the right subtree.

- Insertion
  Insertion in BST involves the comparison of the key values. If the key value is lesser than or equal to root key then go to left subtree, find an empty space following to the search algorithm and insert the data and if the key is greater than root key then go

to right subtree, find an empty space following to the search algorithm and insert the data.

- Deletion
  Case 1- If the node to be deleted is leaf node: If the node to be deleted is a leaf node, then delete it
  Case 2- If the node to be deleted has one child: If the node to be deleted has one child then, delete the node and place the child of the node at the position of the deleted node.
  Case 3- If the node to be deleted has two children: If the node to be deleted has two children then, find the inorder successor or inorder predecessor of the node according to the nearest capable value of the node to be deleted. Delete the inorder successor or predecessor using the above cases. Replace the node with the inorder successor or predecessor.

- Traversals

  There are 4 types of traversals of the Binary Search Tree.

  1. Level Order Traversal
  2. Pre-order Traversal
  3. Inorder Traversal
  4. Post Traversal

Construct a binary search tree from preorder and in order with suitable example

Preorder to Inorder

Page No.
Date

Q.) Construct a Binary tree from Preorder & Inorder

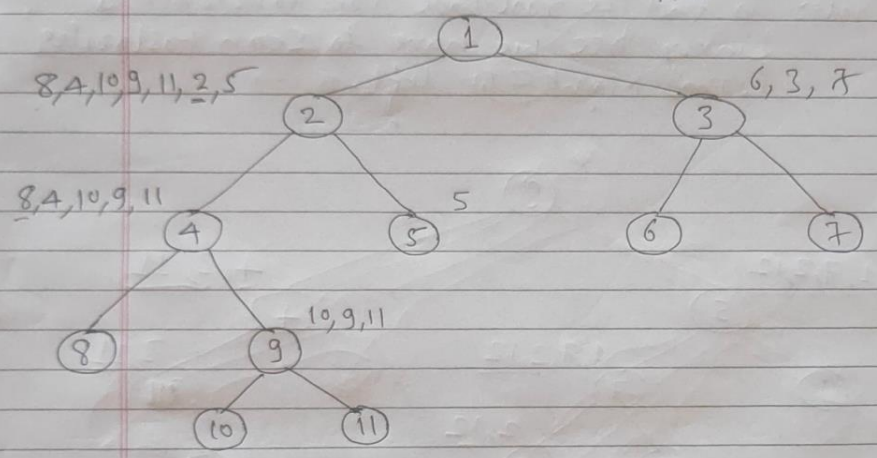
Preorder :- 1, 2, 4, 8, 9, 10, 11, 5, 3, 6, 7 (Root-L-R)

Root

→ Scan from Left to Right

Inorder :- 8, 4, 10, 9, 11, 2, 5, 1, 6, 3, 7 (L-Root-Right)

Root



Note:- In above Example we are selecting or finding out Root Node from preorder because it is first (Root - Left - Right)

& each time we have to scan from Left to right in Preorder

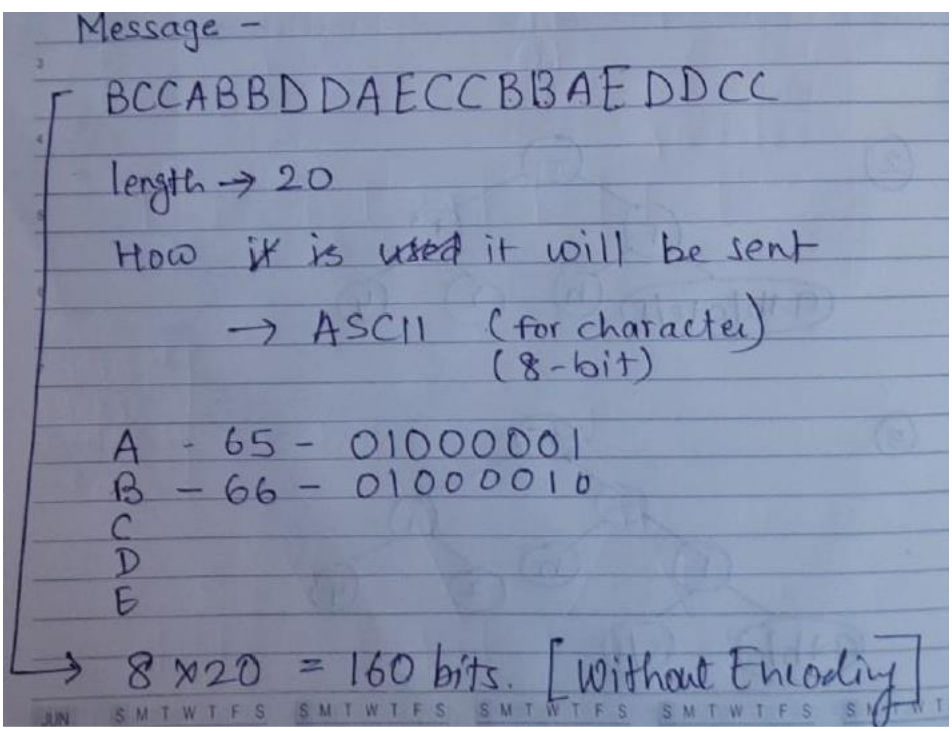
What is Huffman encoding?

Huffman coding is a lossless data compression algorithm. The idea is to assign variable-length codes to input characters, lengths of the assigned codes are based on the frequencies of corresponding characters.

The variable-length codes assigned to input characters are Prefix Codes, means the codes (bit sequences) are assigned in such a way that the code assigned to one character is not the prefix of code assigned to any other character. This is how Huffman Coding makes sure that there is no ambiguity when decoding the generated bitstream.

Let us understand prefix codes with a counter example. Let there be four characters a, b, c and d, and their corresponding variable length codes be 00, 01, 0 and 1. This coding leads to ambiguity because code assigned to c is the prefix of codes assigned to a and b. If the compressed bit stream is 0001, the decompressed output may be “cccd” or “ccb” or “acd” or “ab

Example:



Week 26
176-190

Wednesday 24

Note - Not using every character on keyboard.
we are using only 5 characters (alphabets)

Am i need 8bits to represent only 5 alphabets

OWN Code

Characters	count/freq.		Code
A	3	(3/20)	000
B	5		001
C	6		010
D	4		011
E	2		100

Combinations
— bit - 2
— — bit - 4
— — — bits - 8

* 8 bits are required for 128 characters.

I will use this for encoding msg.

Encoded Message
→ (001)(010) ---

20 x 3 = 60 bits

Chart of Code

Msg 60 bits
Table 55 bits
115 bits.

5 x 8 + 5 x 3
char/ Symbols. codes
40 + 15 = 55 bits

35 - 40% Reduction

Thursday

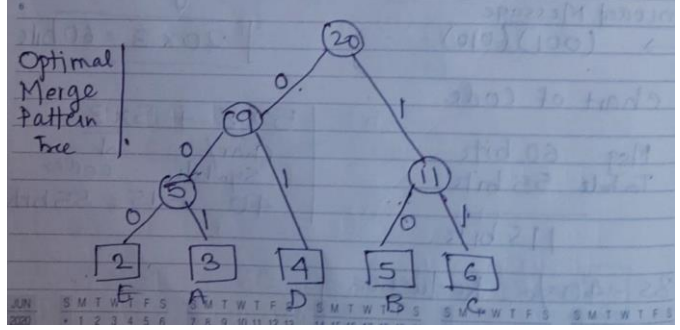
Variable Size Encoding → Optimal.

- We dont have to take fixed size code code for encode

- Small size code for frequently appearing cha

	count/freq.	code	
A	3	001	9
B	5	10	10
C	6	11	12
D	4	01	8
E	2	000	6
	20		45 bits.

Optimal Merge Pattern Tree



B C C A B B D D A
10 11 11 001 10 10 01 01 001

Msg Size - 45 bits
Char / Table - 52 Bits

$\boxed{97 \text{ Bits}}$

$8 \times 5 = 40$
$+ 12 \text{ bits}$
52 bits

From Tree

$$\Sigma (d_i \times f_i)$$
$$= 3\times2 + 3\times3 + 2\times4 + 2\times5 + 2\times6$$

Encoding a Message.

- While encoding read a bit if 1 move to right if 0 move to left.
- Once we reach to leaf, we will get character.
- ~~Next~~ Start from root & repeat procedure

What is AVL Tree?

An AVL tree is a type of binary search tree. Named after it's inventors Adelson, Velskii, and Landis, AVL trees have the property of dynamic self-balancing in addition to all the other properties exhibited by binary search trees.

• AVL trees have an additional guarantee:

1. The difference between the depth of right and left sub-trees cannot be more than one. This difference is called the balance factor.
2. In order to maintain this guarantee, an implementation of an AVL will include an algorithm to rebalance the tree when adding an additional element would upset this guarantee

AVL trees have a worst case lookup, insert, and delete time of O(log n), where n is the number of nodes in the tree. The worst case space complexity is O(n).

• Operations on AVL Tree

The AVL tree is a balancing binary tree, and therefore it follows the same operations we perform in the binary search tree.

1. Insertion
2. Deletion

1. Insertion: The process of insertion is the same as it is executed in the binary search tree. However, there are chances that it may point to a violation in the AVL tree property, and the tree may require balancing. To balance a tree we can apply rotations.
2. Deletion: The process of deletion is the same as it is executed in a binary search tree. It can affect the balance factor of the tree, therefore, we need to utilize different types of rotations to balance the tree.

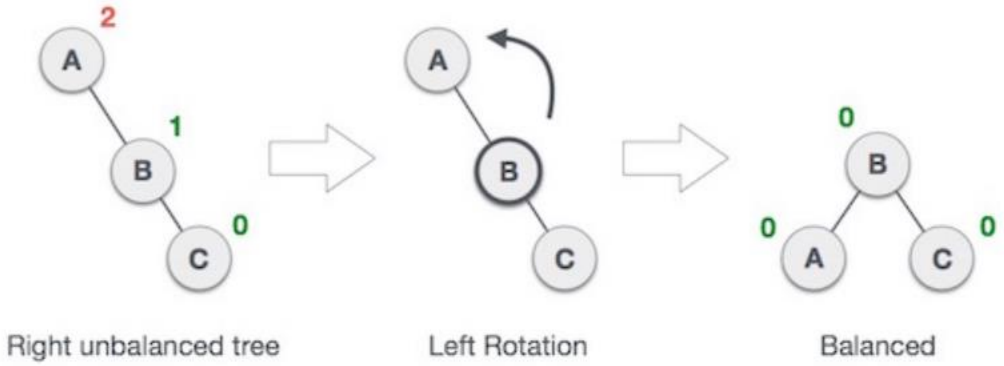
Rotation in AVL tree

To balance itself, an AVL tree may perform the following four kinds of rotations −

- Left rotation
- Right rotation
- Left-Right rotation
- Right-Left rotation

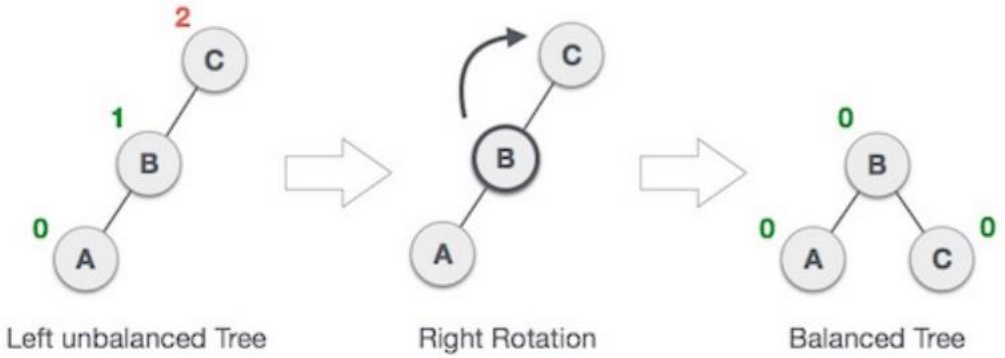
- Left Rotation

  If a tree becomes unbalanced, when a node is inserted into the right subtree of the right subtree, then we perform a single left rotation



- Right Rotation

  AVL tree may become unbalanced, if a node is inserted in the left subtree of the left subtree. The tree then needs a right rotation.



- Left-Right Rotation

  Double rotations are slightly complex version of already explained versions of rotations. To understand them better, we should take note of each action performed while rotation. Let's first check how to perform Left-Right rotation. A left-right rotation is a combination of left rotation followed by right rotation

- Right-Left Rotation

  The second type of double rotation is Right-Left Rotation. It is a combination of right rotation followed by left rotation

Basis of Comparision	B tree	B+ tree
Pointers	All internal and leaf nodes have data pointers	Only leaf nodes have data pointers
Search	Since all keys are not available at leaf, search often takes more time.	All keys are at leaf nodes, hence search is faster and more accurate.
Redundant Keys	No duplicate of keys is maintained in the tree.	Duplicate of keys are maintained and all nodes are present at the leaf.
Insertion	Insertion takes more time and it is not predictable sometimes.	Insertion is easier and the results are always the same.
Deletion	Deletion of the internal node is very complex and the tree has to undergo a lot of transformations.	Deletion of any node is easy because all node are found at leaf.
Leaf Nodes	Leaf nodes are not stored as structural linked list.	Leaf nodes are stored as structural linked list.
Access	Sequential access to nodes is not possible	Sequential access is possible just like linked list
Height	For a particular number nodes height is larger	Height is lesser than B tree for the same number of nodes

Application	B-Trees used in Databases, Search engines	B+ Trees used in Multilevel Indexing, Database indexing
Number of Nodes	Number of nodes at any intermediary level ‘l’ is 2l.	Each intermediary node can have n/2 to n children.

Module 5

Graph terminologies

1. Path: A path is the sequence of nodes that is followed to reach some terminal vertex X from the initial vertex Y.

2. Closed path: A path is a closed path if the initial vertex is the same as the terminal vertex.

3. Simple path: If all the vertices in the graph are distinct (exception V0=Vn), then path P is a closed simple path.

4. Cycle: A cycle is a path that has no repeated edges or vertices except the first and last vertices.

5. Connected graph: In a connected graph each pair of nodes has a path between them i.e. we can reach any vertex by starting from any vertex in the graph. There are no isolated nodes in the connected graph.

6. Complete graph: A complete graph is the one in which each pair of nodes has a direct path between them. A complete graph has n(n-1)/2 edges where n is the number of vertices in the graph.

7. Weighted graph: In a weighted graph, each edge is assigned with a data called weight. The weight of an edge E is given as W(E). Weight must be a positive value that defines the cost of traversing the edge R.

8. Digraph: A directed graph is sometimes referred to as a digraph.

9. Loop: An edge that has the same endpoints is a loop.

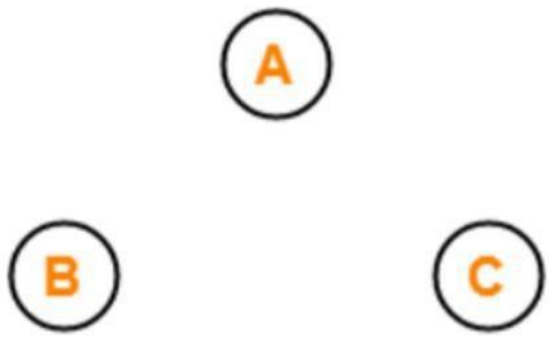
10. Adjacent nodes: If two nodes A and B are directly connected via an edge E, then the vertices A and B are adjacent nodes.

11. Degree of the node: A degree of a node is the number of edges associated with that node. A node with degree 0 is an isolated node.

Types of Graphs

• Null Graph

The Null Graph is also known as the order zero graph. The term “null graph” refers to a graph with an empty edge set. In other words, a null graph has no edges, and the null graph is present with only isolated vertices in the graph.

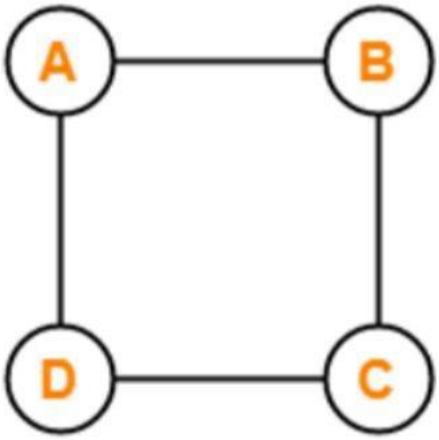


• Trivial Graph

A graph is called a trivial graph if it has only one vertex present in it. The trivial graph is the smallest possible graph that can be created with the least number of vertices that is one vertex only.

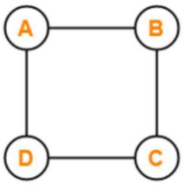
• Non-Directed Graph

A graph is called a non-directed graph if all the edges present between any graph nodes are non-directed. By non-directed edges, we mean the edges of the graph that cannot be determined from the node it is starting and at which node it is ending. All the edges for a graph need to be non-directed to call it a non-directed graph. All the edges of a non-directed graph don't have any direction.



• Directed Graph

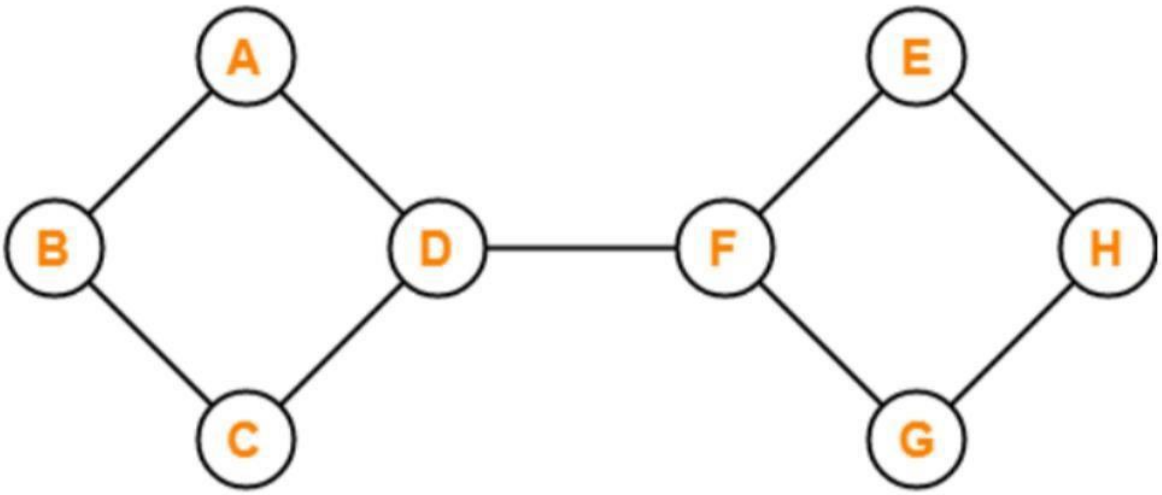
Another name for the directed graphs is digraphs. A graph is called a directed graph or digraph if all the edges present between any vertices or nodes of the graph are directed or have a defined direction. By directed edges, we mean the edges of the graph that have a direction to determine from which node it is starting and at which node it is ending.



(Arrows are missing over here, just add arrows)

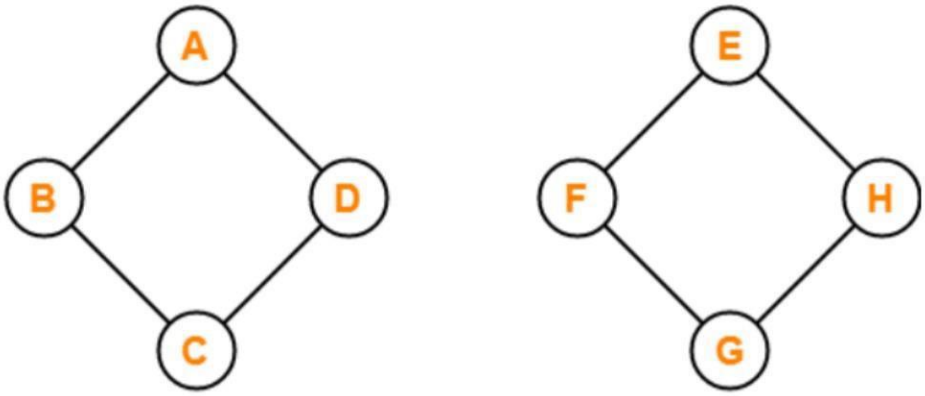
• Connected Graph

For a graph to be labelled as a connected graph, there must be at least a single path between every pair of the graph's vertices. In other words, we can say that if we start from one vertex, we should be able to move to any of the vertices that are present in that particular graph, which means there exists at least one path between all the vertices of the graph.



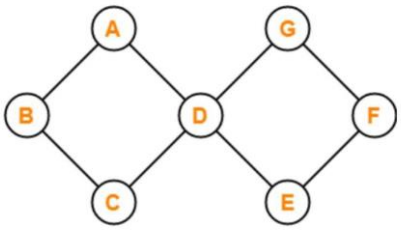
• Disconnected Graph

A graph is said to be a disconnected graph where there does not exist any path between at least one pair of vertices. In other words, we can say that if we start from any one of the vertices of the graph and try to move to the remaining present vertices of the graph and there exists not even a single path to move to that vertex, then it is the case of the disconnected graph. If any one of such a pair of vertices doesn't have a path between them, it is called a disconnected graph.



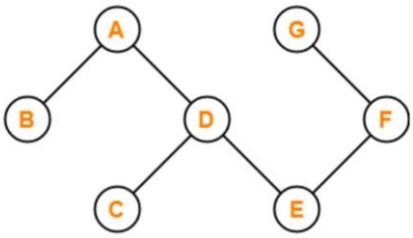
• Cyclic Graph

For a graph to be called a cyclic graph, it should consist of at least one cycle. If a graph has a minimum of one cycle present, it is called a cyclic graph.



• Acyclic Graph

A graph is called an acyclic graph if zero cycles are present, and an acyclic graph is the complete opposite of a cyclic graph.



Topological Sorting

A topological sort or topological ordering of a directed graph is a linear ordering of its vertices in which u occurs before v in the ordering for every directed edge uv from vertex u to vertex v. For example, the graph's vertices could represent jobs to be completed, and the edges could reflect requirements that one work must be completed before another. In this case, a topological ordering is just a legitimate task sequence. A topological sort is a graph traversal in which each node v is only visited after all of its dependencies have been visited. If the graph contains no directed cycles, then it is a directed acyclic graph. Any DAG has at least one topological ordering, and there exist techniques for building topological orderings in linear time for any DAG. Topological sorting has many applications, particularly in ranking issues like the feedback arc set.

Application of Topological sorting

1. Finding cycle in a graph
2. Operation System deadlock detection
3. Dependency resolution
4. Sentence Ordering
5. Critical Path Analysis
6. Course Schedule problem
7. Other applications like manufacturing workflows, data serialization and context-free grammar.

Module 6

What is Hashing?

- Hashing in the data structure is a technique of mapping a large chunk of data into small tables using a hashing function.
- It is also known as the message digest function.
- It is a technique that uniquely identifies a specific item from a collection of similar items.
- It uses hash tables to store the data in an array format. Each value in the array has been assigned a unique index number.
- Hash tables use a technique to generate these unique index numbers for each value stored in an array format.
- This technique is called the hash technique.
- The hash table is basically the array of elements and the hash techniques of search are performed on a part of the item i.e. key. Each key has been mapped to a number, the range remains from 0 to table size 1
- You only need to find the index of the desired item, rather than finding the data.
- With indexing, you can quickly scan the entire list and retrieve the item you wish.
- Indexing also helps in inserting operations when you need to insert data at a specific location.
- No matter how big or small the table is, you can update and retrieve data within seconds.

What are Hash functions?

A Hash Function is a function that converts a given numeric or alphanumeric key to a small practical integer value. The mapped integer value is used as an index in the hash table. In simple terms, a hash function maps a significant number or string to a small integer that can be used as the index in the hash table. The pair is of the form (key, value), where for a given key, one can find a value using some kind of a "function" that maps keys to values. The key for a given object can be calculated using a function called a hash function.

Types of Hash functions:

1. Division Method.
2. Mid Square Method.
3. Folding Method.
4. Multiplication Method.

What are the Collision Resolution Techniques?

1. Separate chaining: This method involves making a linked list out of the slot where the collision happened, then adding the new key to the list. Separate chaining is the term used to describe how this connected list of slots resembles a chain. It is more frequently utilized when we are unsure of the number of keys to add or remove.

   Advantages of separate chaining

   - It is easy to implement.
   - The hash table never fills full, so we can add more elements to the chain.
   - It is less sensitive to the function of the hashing.

Open addressing: To prevent collisions in the hashing table open, addressing is employed as a collision-resolution technique. No key is kept anywhere else besides the hash table. As a result, the hash table's size is never equal to or less than the number of keys. Additionally known as closed hashing.

Advantages:

- Calculating this is simple.
- To prevent clustering and load factor, open addressing calls for extra caution.